

app, mic edge scenario

Overview

This document defines the flow that an app must handle when a USB microphone currently open via the `openMicrophone()` API is unplugged. The service detects USB disconnection and notifies `getMicrophones` subscribers with an updated list, but does **not** automatically close the open microphone or switch to another one. **The app is responsible for handling this.**

Prerequisites

- `keyType: "karaoke"` (the only `keyType` that opens a USB microphone as a loopback)
- `openMicrophone()` must always be called with `subscribe: true`. (So the service can automatically call `closeMicrophoneImpl` on app crash or normal exit.)
- `startMicrophone()` must be called immediately after a successful `openMicrophone()`.
- `stopMicrophone()` must be called immediately before `closeMicrophone()`.



- please always check `microphoneEnabled == true` before calling `openMicrophone()`

Setup: Initial Subscriptions

Register the following subscriptions when the app starts.

```
// Detect microphone list changes
luna-send -i -f luna://com.webos.service.microphone/getMicrophones \
'{"subscribe": true, "keyType": "karaoke}"'
```

The `getMicrophones` subscription receives an updated list via `notifyMicListChange()` whenever a USB device is connected or disconnected. The app detects hotplug events by checking whether the currently open `cardId` is present in the received list.

Scenario A: Auto-switch to Another USB Microphone

State

- USB-A (`card-usb-A`) → currently open (active)
- USB-B (`card-usb-B`) → connected but not in use
- `preferredMicrophone` not set (or set to USB-A)

Flow

Since `preferredMicrophone` is not set, `resolveMicrophone()` returns the first available USB microphone from the ALSA list (USB-B).

[App]	[Microphone Service]
getMicrophones(subscribe:true, keyType:karaoke)	
{returnValue:true, subscribed:true,	
micList:[card-usb-A, card-usb-B], ...}	
openMicrophone({deviceType:"usb",	
keyType:"karaoke", subscribe:true})	
{returnValue:true, socketPath:"..."}	active_microphone_id_ = "card-usb-A"
startMicrophone({deviceType:"usb",	
keyType:"karaoke"})	
{returnValue:true}	
[User unplugs USB-A]	
getMicrophones subscription notification	notifyMicListChange() called
{micList:[card-usb-B],	(list without USB-A)
activeMicrophones:"card-usb-A", ...}	
[App: activeMicrophones not in micList disconnect]	
stopMicrophone({deviceType:"usb",	
keyType:"karaoke"})	
{returnValue:true}	
closeMicrophone({deviceType:"usb",	
keyType:"karaoke"})	
{returnValue:true}	active_microphone_id_ = ""
[App: card-usb-B in micList attempt switch]	
openMicrophone({deviceType:"usb",	
keyType:"karaoke", subscribe:true})	resolveMicrophone() "card-usb-B"
{returnValue:true, socketPath:"..."}	active_microphone_id_ = "card-usb-B"
startMicrophone({deviceType:"usb",	
keyType:"karaoke"})	
{returnValue:true}	
getMicrophones subscription notification	notifyMicListChange() called
{micList:[card-usb-B],	(activeMicrophones updated)
activeMicrophones:"card-usb-B", ...}	

Scenario B: No Microphone Available → Auto Stop

State

- USB-A (card-usb-A) → the only USB microphone, currently open

Flow

[App]	[Microphone Service]
openMicrophone({deviceType:"usb",	
keyType:"karaoke", subscribe:true})	
{returnValue:true, socketPath:"..."}	
startMicrophone({deviceType:"usb",	
keyType:"karaoke"})	
{returnValue:true}	
[User unplugs USB-A]	
getMicrophones subscription notification	
{micList:[], activeMicrophones:"card-usb-A", ...}	
[App: activeMicrophones not in micList,	
micList is empty no microphone available]	
stopMicrophone({deviceType:"usb",	
keyType:"karaoke"})	
{returnValue:true}	
closeMicrophone({deviceType:"usb",	
keyType:"karaoke"})	
{returnValue:true}	active_microphone_id_ = ""
[App: disable microphone UI and wait]	

Scenario C: New USB Microphone Connected

A new microphone connection is detected when a cardId not present in the previous micList appears in the getMicrophones subscription notification. The app must keep a snapshot of the previous micList and compare it against each incoming notification to identify newly added entries.

C-1: The newly connected microphone is the preferredMicrophone

If the connected microphone matches preferredMicrophone, switch to it regardless of what is currently active.

C-1-a: Another microphone is currently open

State

- USB-A (card-usb-A) → currently open (active), not preferred
- USB-B (card-usb-B) → set as preferredMicrophone but not connected
- USB-B is newly connected

[App]	[Microphone Service]
[USB-B connected]	
getMicrophones subscription notification	notifyMicListChange() called
{micList:[card-usb-A, card-usb-B],	
activeMicrophones:"card-usb-A",	
preferredMicrophone:"card-usb-B", ...}	
[App: newly added card-usb-B == preferredMicrophone	
switch required]	
stopMicrophone({deviceType:"usb",	
keyType:"karaoke"})	
{returnValue:true}	
closeMicrophone({deviceType:"usb",	
keyType:"karaoke"})	
{returnValue:true}	active_microphone_id_ = ""
openMicrophone({deviceType:"usb",	
keyType:"karaoke", subscribe:true}) resolveMicrophone() "card-usb-B"	
{returnValue:true, socketPath:"..."}	active_microphone_id_ = "card-usb-B"
startMicrophone({deviceType:"usb",	
keyType:"karaoke"})	
{returnValue:true}	
getMicrophones subscription notification	
{micList:[card-usb-A, card-usb-B],	
activeMicrophones:"card-usb-B", ...}	

C-1-b: No microphone currently open (e.g. waiting after Scenario B)

State

- No active microphone
- USB-A (card-usb-A) → set as preferredMicrophone but not connected
- USB-A is newly connected

[App]	[Microphone Service]
[USB-A connected]	
getMicrophones subscription notification	notifyMicListChange() called
{micList:[card-usb-A],	
activeMicrophones:"",	
preferredMicrophone:"card-usb-A", ...}	
[App: newly added card-usb-A == preferredMicrophone	
open]	
openMicrophone({deviceType:"usb",	
keyType:"karaoke", subscribe:true}) resolveMicrophone() "card-usb-A"	
{returnValue:true, socketPath:"..."}	active_microphone_id_ = "card-usb-A"
startMicrophone({deviceType:"usb",	
keyType:"karaoke"})	
{returnValue:true}	

C-2: The newly connected microphone is NOT the preferredMicrophone

C-2-a: Another microphone is currently open → ignore

State

- USB-A (card-usb-A) → currently open (active)
- preferredMicrophone not set (or set to USB-A)
- USB-B (card-usb-B) → non-preferred microphone newly connected

[App]	[Microphone Service]
[USB-B connected]	
getMicrophones subscription notification	notifyMicListChange() called
{micList:[card-usb-A, card-usb-B],	
activeMicrophones:"card-usb-A", ...}	
[App: newly added card-usb-B preferredMicrophone,	
current microphone still active no action]	

C-2-b: No microphone currently open and preferredMicrophone not set → open

State

- No active microphone
- preferredMicrophone not set
- USB-A (card-usb-A) → newly connected

When preferredMicrophone is not set, resolveMicrophone() returns the first available USB microphone.

[App]	[Microphone Service]
[USB-A connected]	
getMicrophones subscription notification	notifyMicListChange() called
{micList:[card-usb-A],	
activeMicrophones:"",	
preferredMicrophone:"", ...}	
[App: no active mic, no preferred mic	
open the connected microphone]	
openMicrophone({deviceType:"usb",	
keyType:"karaoke", subscribe:true}) resolveMicrophone() "card-usb-A"	
{returnValue:true, socketPath:"..."}	active_microphone_id_ = "card-usb-A"
startMicrophone({deviceType:"usb",	
keyType:"karaoke"})	
{returnValue:true}	

C-2-c: No microphone currently open and preferredMicrophone is set but not connected → attempt open if microphoneEnabled==true

State

- No active microphone
- preferredMicrophone: "card-usb-B" set but not connected
- USB-A (card-usb-A) → non-preferred microphone newly connected

When preferredMicrophone is not connected, resolveMicrophone() skips the preferred and returns the first connected USB microphone as a fallback.

[App]	[Microphone Service]
[USB-A connected]	
getMicrophones subscription notification	
{micList:[card-usb-A],	
activeMicrophones:"",	
preferredMicrophone:"card-usb-B", ...}	
[App: card-usb-A preferredMicrophone("card-usb-B"),	
microphoneEnabled==true attempt open]	
openMicrophone({deviceType:"usb",	
keyType:"karaoke", subscribe:true}) resolveMicrophone() "card-usb-A"	
{returnValue:true, socketPath:"..."} active_microphone_id_ = "card-usb-A"	
startMicrophone({deviceType:"usb",	
keyType:"karaoke"})	
{returnValue:true}	

Note: If microphoneEnabled==false, take no action and wait for the preferred microphone to connect.

Key Rules Summary

Rule	Description
Subscribe to getMicrophones	The only way to detect USB connect/disconnect. A disconnect is identified when activeMicrophones is no longer present in micList.
stopMicrophone first	Must be called before closeMicrophone().
Explicit closeMicrophone required	Even though the service closes internally when the subscription is cancelled, the app must explicitly call closeMicrophone() .
startMicrophone after open	Must be called after a successful openMicrophone().

Minimum API Call Sequence (normal switch)

```

Disconnect detected
stopMicrophone(usb, karaoke)
closeMicrophone(usb, karaoke)
[optional] setPreferredMicrophone(next cardId or "")
openMicrophone(usb, karaoke, subscribe:true) continue only on success
startMicrophone(usb, karaoke)

```